

Admin

Apple visit today after lecture

Interrupts

Final touch for completing full system!

Today

- Exceptional control flow

 - Suspend, jump to different code, then resume

 - How to do this safely and correctly

- Low-level mechanisms

Next lecture

- Using interrupts as client

- Coordination of activity

 - Concurrency, multiple handlers, shared data



libmango

gpio ✓

timer ✓

uart ✓

strings ✓

printf ✓

malloc ✓

keyboard ✓

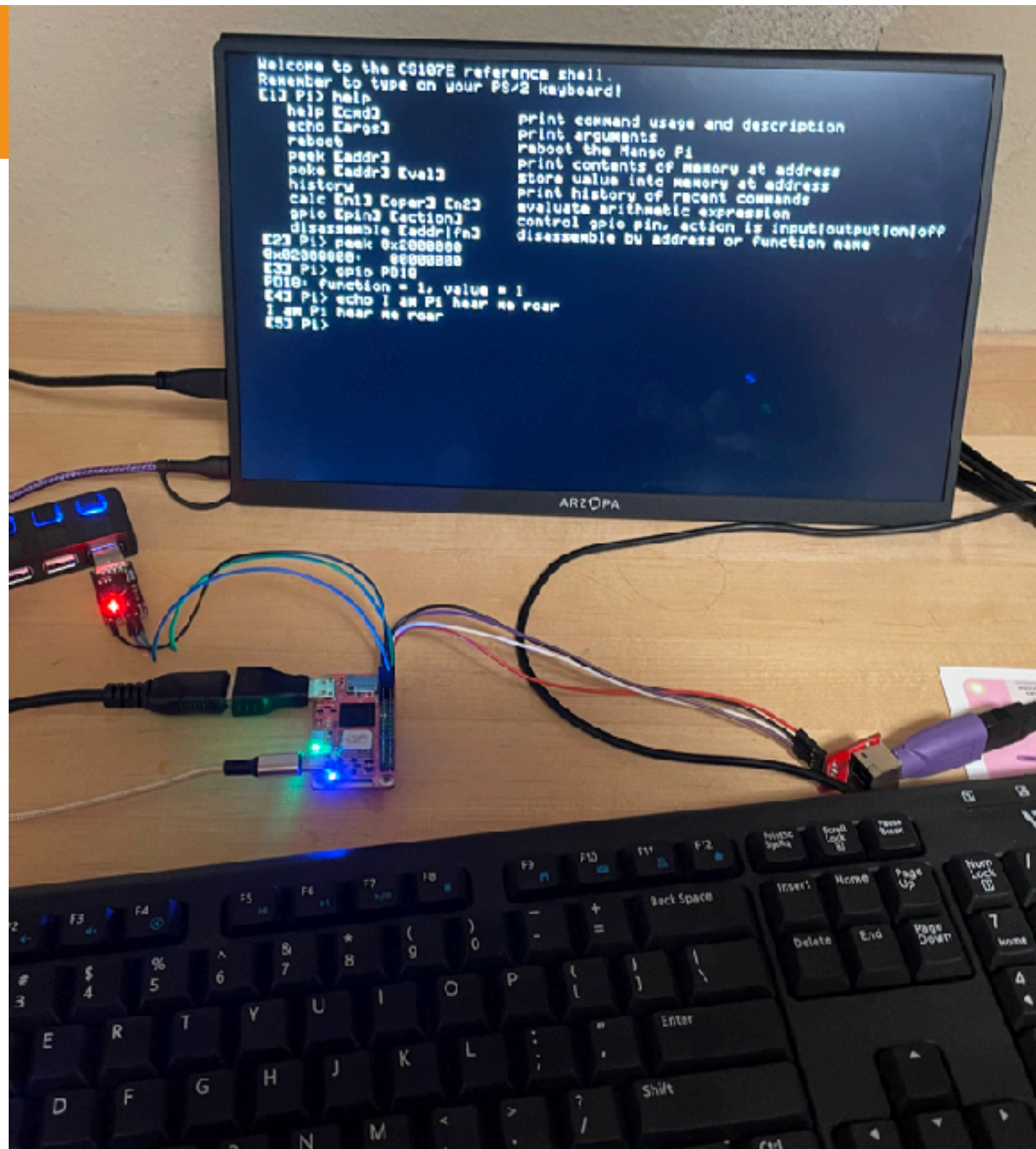
shell ✓

fb ✓

gl ✓

console ✓

→ **interrupts**



Blocking I/O

```
while (1) {  
    char ch = keyboard_read_next();  
    update_screen();  
}
```

How long does it take to send a scan code?

11 bits at PS/2 bit rate 15kHz

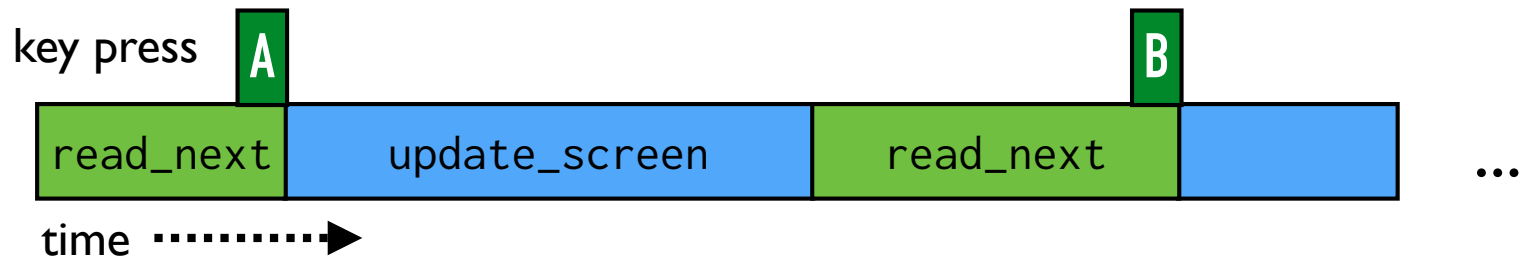
How long does it take to update the screen?

What could go wrong?

Blocking I/O

```
while (1) {  
    char ch = keyboard_read_next();  
    update_screen();  
}
```

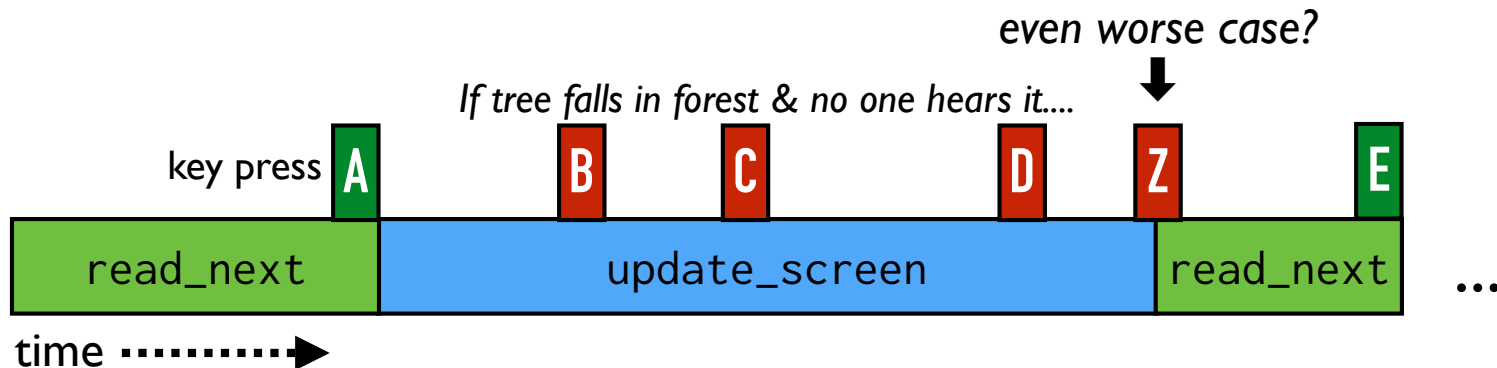
"Good" (i.e. lucky) scenario works out...



Blocking I/O

```
while (1) {  
    char ch = keyboard_read_next();  
    update_screen();  
}
```

Bad scenario does not!



The Problem

Ongoing, long-running tasks (graphics, simulations, applications) keep CPU occupied, but ...

When an external event arises, need to respond quickly.

Consider: Why does your cell ring/buzz? What would you have to do to see a text arrive if it didn't?

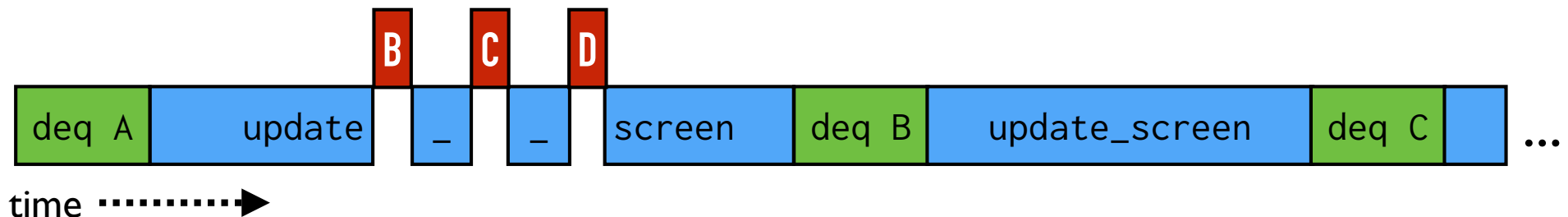
Concurrency

How to juggle doing more than one thing at once?

```
while (1) {  
    dequeue scandcode from queue  
    update_screen();  
}
```

whenever scandcode arrives,
enqueue it

scancodes buffered for later processing



Interrupts to the rescue!

Processor pauses current execution, switch to code that handles interrupt, switch back and resume execution

- Asynchronous external event (peripherals, timer)

- Synchronous exception (invalid address, illegal instruction)

Critical for responsive systems, hosted OS

Interrupts are essential and powerful, but getting them right requires using everything you've learned:

- Architecture, assembly, linking, memory, C, peripherals, ...

code/button-blocking
code/button-interrupt

Interrupted control flow

```
static volatile int gCount;
```

```
void update_screen(void) {  
    console_clear();  
    for (int i = 0; i < N;  
        console_printf("%d"  
    }
```

```
void button_pressed(void *aux_data) {  
    gCount++;  
    gpio_interrupt_clear(BUTTON);  
}
```

```
9    9    9    9    9  
9    9   10   10   10  
10   10
```

Suspend current activity, execute other code, then resume, ... this will be tricky!

Interrupt mechanics

Somewhat analogous to function call

- Suspend currently executing code, save state
- Jump to handler code, process interrupt
- When finished, restore state and resume

Must adhere to conventions to avoid stepping on each other

- Consider: processor state, register use, memory
- Hardware support helps out

C abstraction ...

Asm understanding!

```
update_screen:
    add    sp,sp,-16
    sd     ra,8(sp)
    sd     s0,0(sp)
    add    s0,sp,16
    jal    0x400006c4 <console_clear>
    lui    a0,0x40008
    add    a0,a0,496
    jal    <console_printf>
    li     s1,0
    j      <update_screen+68>
    lui    a5,0x40022
    lw     a1,-760(a5)
    . . .
```

Interrupt!



```
button_pressed:
    add    sp,sp,-16
    sd     ra,8(sp)
    sd     s0,0(sp)
    add    s0,sp,16
    lui    a4,0x40022
    lw     a5,-760(a4)
    addw   a5,a5,1
    sw     a5,-760(a4)
    li     a0,1293
    jal    <gpio_interrupt_clear>
    ld     ra,8(sp)
    ld     s0,0(sp)
    add    sp,sp,16
    ret
```

How to go from `update_screen` to `button_pressed` and back again?

- 1) pause, preserve state
- 2) jump to handler, execute
- 3) restore state, resume

The RISC-V Instruction Set Manual
Volume II: Privileged Architecture
Document Version 20190608-Priv-MSU-Ratified

Editors: Andrew Waterman¹, Krste Asanović^{1,2}

¹SiFive Inc.,

²CS Division, EECS Department, University of California, Berkeley
andrew@sifive.com, krste@berkeley.edu

June 8, 2019

<https://cs107e.github.io/readings/riscv-privileged-20190608-1.pdf>

Terminology

Exception

event that disrupts normal program flow

forces transfer of control to trap handler

Interrupt

- external asynchronous event
- hardware-generated signals from devices (timer, uart, gpio)
- not caused by executing instruction, timing is unpredictable
- interrupted exception is innocent bystander, will resume

Fault/abort

- error raised when executing specific instruction (invalid access, illegal instruction)
- synchronous, predictable
- executing instruction is the one in error, may be possible to fix and retry

All exceptions transfer control to single trap handler

Processor privilege levels

Machine Mode (M-mode)	Direct hardware access All CSRs accessible No blocks/restrictions	Entered on reset Bare-metal runs in M-mode!
Supervisor Mode (S-mode)	Some CSRs restricted Page tables, virtual memory	OS kernel would run here Supervises user-mode access to memory/hardware
User Mode (U-mode)	Most restricted No access to CSRs	Regular programs Request to kernel/supervisor as needed

Each privilege level has separate interrupt control
Can elevate/delegate to redirect to different privilege level

All our code runs in M-mode
We only use the M-mode interrupt control

Interrupt CSRs

Machine Trap Setup

mstatus	Machine status register.
misa	ISA and extensions
medeleg	Machine exception delegation register.
mideleg	Machine interrupt delegation register.
mie	Machine interrupt-enable register.
mtvec	Machine trap-handler base address.
mcounteren	Machine counter enable.

Global master switch

Config which interrupt types are enabled

Function pointer

Machine Trap Handling

mscratch	Scratch register for machine trap handlers.
mepc	Machine exception program counter.
mcause	Machine trap cause.
mtval	Machine bad address or instruction.
mip	Machine interrupt pending.

Address of instruction that was interrupted

What event caused trap?

Bad val when trap caused by error

<https://cs107e.github.io/readings/riscv-privileged-20190608-1.pdf>

Config M-mode external interrupts

```
CSR_WRITE(mtvec, _trap_handler);    // set function pointer
CSR_SET_BIT(mie, 1 << 11);          // enable external interrupts
CSR_SET_BIT(mstatus, 1 << 3);        // global switch on
```


Interrupt, hardware-side

External event triggers interrupt line. Processor response:

- Complete current instruction, hold off on next

- Saves `pc` into `mepc` (address of insn that would have executed)

- Disables further interrupts

- Jump to trap handler address (`mtvec`)

Software takes over, trap handler is now executing

- Handler must save/restore any registers it uses

- Handler processes interrupt and clears it

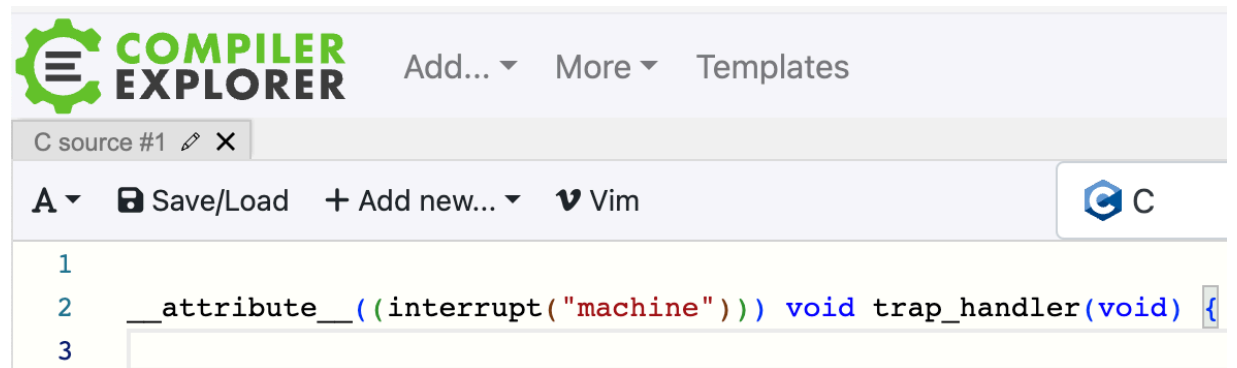
- Handler ends with `mret` instruction

- Reverses hardware actions: reenables interrupts, `pc = mepc`
thereby restoring state and resuming at interrupted `pc`

Interrupt, software-side

```
__attribute__((interrupt("machine")))  
void trap_handler(void)  
{  
    . . . .  
}
```

How does interrupt attribute change the generated code? Let's find out!



<https://gcc.godbolt.org/z/fPGdz4MPP>

```
CSR_WRITE(mtvec, trap_handler);    // store fn ptr to mtvec CSR
```

```
la      t0, trap_handler  
csrw    mtvec, t0
```

Interrupts (so far)

Hardware support for exceptional control flow

- Pause current, force to trap handler, then resume

Top-level interrupts system configuration

- mtvec CSR to set trap handler address
- mstatus, mie CSRs to enable interrupts, switch on

Transfer of control to enter/exit interrupt code

- Assembly to preserve registers
- Call into C code
- Assembly to restore registers, resume interrupted code

Next Lecture

Client use of interrupts

Which interrupts are supported and how to configure
(events from gpio, hstimer, uart, ...)

Dispatch to specific handler per event source

Steps to process/clear event

Writing safe interrupt handlers

How to share state without conflicts